

Atelier Django

Mise en place d'un système d'authentification dans une application Django

Inscription, connexion, déconnexion et page profil

Objectifs de l'atelier

À la fin de cet atelier, l'étudiant sera capable de :

- ajouter une application **accounts** dans un projet Django existant ;
- utiliser le système d'authentification intégré de Django ;
- créer une page d'inscription ;
- créer une page de connexion ;
- gérer la déconnexion ;
- créer une page profil protégée ;
- réutiliser le template **layout.html** déjà existant dans l'application **products** ;
- intégrer l'authentification dans un projet e-commerce.

1 Contexte de l'atelier

Dans cet atelier, le projet Django existe déjà. Il s'agit d'un projet e-commerce contenant une application appelée **products**.

L'application **products** permet déjà d'afficher :

- la liste des produits ;
- le détail d'un produit ;
- les images des produits ;
- les produits et les catégories dans l'administration Django.

L'objectif de cet atelier est d'ajouter une nouvelle application appelée **accounts** pour gérer :

- l'inscription des utilisateurs ;
- la connexion ;
- la déconnexion ;
- l'affichage du profil utilisateur ;
- la protection de certaines pages.

2 Architecture actuelle du projet

L'architecture actuelle du projet est la suivante :

```
ecommerce/  
  
    ecommerce/  
        settings.py  
        urls.py  
        asgi.py  
        wsgi.py
```

```
images/

products/
  migrations/
  templates/
    layout.html
    product_detail.html
    product_list.html
  admin.py
  apps.py
  models.py
  tests.py
  urls.py
  views.py

db.sqlite3
docker-compose.yaml
manage.py
myenv/
```

Après l'atelier, l'architecture deviendra :

```
ecommerce/

ecommerce/
  settings.py
  urls.py
  asgi.py
  wsgi.py

images/

products/
  templates/
    layout.html
    product_detail.html
    product_list.html
  urls.py
  views.py
  ...

accounts/
  templates/
    registration/
      login.html
      signup.html
      profile.html
  forms.py
  urls.py
  views.py
  ...

db.sqlite3
docker-compose.yaml
manage.py
myenv/
```

Remarque

Dans cet atelier, il ne faut pas recréer le projet Django. Le projet existe déjà. Il faut seulement ajouter une nouvelle application appelée **accounts**.

3 Partie 1 : Se placer dans le dossier du projet

Ouvrir le terminal dans le dossier qui contient le fichier `manage.py`.

Dans votre cas, il faut être dans ce dossier :

```
ecommerce/
```

Vérifier que le fichier `manage.py` existe :

```
dir
```

Sous Linux ou macOS :

```
ls
```

Vous devez voir des éléments comme :

```
manage.py
db.sqlite3
ecommerce
products
images
docker-compose.yaml
myenv
```

4 Partie 2 : Activer l'environnement virtuel

Sous Windows :

```
myenv\Scripts\activate
```

Sous Linux ou macOS :

```
source myenv/bin/activate
```

5 Partie 3 : Créer l'application accounts

Exécuter la commande suivante :

```
python manage.py startapp accounts
```

Cette commande ajoute une nouvelle application appelée **accounts** à côté de l'application **products**.

Définition

Une application Django représente un module fonctionnel. Ici, l'application **accounts** va contenir le code lié aux comptes utilisateurs : inscription, connexion, déconnexion et profil.

6 Partie 4 : Ajouter accounts dans settings.py

Ouvrir le fichier suivant :

```
ecommerce/settings.py
```

Dans la variable `INSTALLED_APPS`, ajouter `accounts` sans supprimer `products`.

Exemple :

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
  
    'products',  
    'accounts',  
]
```

Remarque

Il ne faut pas supprimer l'application `products`. Elle contient déjà les produits, les catégories, les vues et les templates du projet e-commerce.

7 Partie 5 : Vérifier la configuration des templates

Toujours dans le fichier :

```
ecommerce/settings.py
```

Vérifier que la configuration des templates contient bien :

```
'APP_DIRS': True,
```

Exemple :

```
TEMPLATES = [  
    {  
        'BACKEND': 'django.template.backends.django.DjangoTemplates',  
        'DIRS': [],  
        'APP_DIRS': True,  
        'OPTIONS': {  
            'context_processors': [  
                'django.template.context_processors.request',  
                'django.contrib.auth.context_processors.auth',  
                'django.contrib.messages.context_processors.messages',  
            ],  
        },  
    },  
]
```

Définition

L'option `APP_DIRS=True` permet à Django de chercher automatiquement les templates dans les dossiers `templates` des applications.

Dans notre cas, Django peut donc trouver :

- `products/templates/layout.html`
- `products/templates/product_list.html`
- `products/templates/product_detail.html`
- `accounts/templates/registration/login.html`
- `accounts/templates/registration/signup.html`
- `accounts/templates/registration/profile.html`

À la fin du fichier `settings.py`, ajouter :

```
LOGIN_REDIRECT_URL = "profile"
LOGOUT_REDIRECT_URL = "product_list"
LOGIN_URL = "login"
```

Définition

`LOGIN_REDIRECT_URL` indique la page vers laquelle l'utilisateur est redirigé après une connexion réussie.

`LOGOUT_REDIRECT_URL` indique la page vers laquelle l'utilisateur est redirigé après la déconnexion.

`LOGIN_URL` indique la page de connexion utilisée lorsqu'un utilisateur non connecté essaye d'accéder à une page protégée.

8 Partie 6 : Créer le formulaire d'inscription

Dans le dossier `accounts`, créer un fichier :

```
accounts/forms.py
```

Ajouter le code suivant :

```
from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User

class RegisterForm(UserCreationForm):
    email = forms.EmailField(required=True, label="Adresse email")

    class Meta:
        model = User
        fields = ["username", "email", "password1", "password2"]
```

Définition

`UserCreationForm` est un formulaire fourni par Django pour créer un utilisateur. Dans cet atelier, nous l'avons personnalisé pour ajouter le champ `email`.

9 Partie 7 : Créer les vues de l'application accounts

Ouvrir le fichier :

```
accounts/views.py
```

Remplacer son contenu par :

```
from django.shortcuts import render, redirect
from django.contrib.auth import login
from django.contrib.auth.decorators import login_required
from django.contrib import messages

from .forms import RegisterForm

def signup(request):
    if request.method == "POST":
        form = RegisterForm(request.POST)

        if form.is_valid():
            user = form.save()
            login(request, user)
            messages.success(request, "Votre compte a été créé avec succès.")
            return redirect("profile")
        else:
            form = RegisterForm()

    return render(request, "registration/signup.html", {"form": form})

@login_required
def profile(request):
    return render(request, "registration/profile.html")
```

Définition

La vue `signup` permet de créer un nouveau compte utilisateur. Après la création du compte, l'utilisateur est connecté automatiquement grâce à la fonction `login`.

Définition

Le décorateur `@login_required` permet de protéger une vue. Dans notre cas, la page `profile` est accessible uniquement aux utilisateurs connectés.

10 Partie 8 : Créer les URLs de accounts

Dans le dossier `accounts`, créer un fichier :

```
accounts/urls.py
```

Ajouter le code suivant :

```
from django.urls import path
from . import views

urlpatterns = [
```

```
path("signup/", views.signup, name="signup"),
path("profile/", views.profile, name="profile"),
]
```

11 Partie 9 : Modifier les URLs principales du projet

Ouvrir le fichier :

```
ecommerce/urls.py
```

Votre fichier contient déjà les routes de l'administration, des produits et des images. Il faut conserver ces parties et ajouter les routes de l'authentification.

Remplacer le contenu par cette version :

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static
from django.views.generic import RedirectView

urlpatterns = [
    path('admin/', admin.site.urls),

    path('', RedirectView.as_view(pattern_name='product_list', permanent=False)),

    path('products/', include("products.urls")),

    path('accounts/', include("accounts.urls")),
    path('accounts/', include("django.contrib.auth.urls")),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Remarque

La ligne suivante redirige la racine du site vers la liste des produits :

```
path('', RedirectView.as_view(pattern_name='product_list', permanent=False)),
```

Ainsi, si l'utilisateur ouvre :

```
http://127.0.0.1:8000/
```

il sera redirigé automatiquement vers :

```
http://127.0.0.1:8000/products/
```

Cela évite de créer une page `index.html`.

Définition

La ligne suivante active les URLs d'authentification fournies par Django :

```
path('accounts/', include("django.contrib.auth.urls")),
```

Elle ajoute automatiquement plusieurs routes, par exemple :

- /accounts/login/
- /accounts/logout/
- /accounts/password_change/
- /accounts/password_reset/

12 Partie 10 : Vérifier les URLs de products

Votre fichier :

```
products/urls.py
```

doit rester comme suit :

```
from django.urls import path

from . import views

urlpatterns = [
    path("", views.product_list, name="product_list"),
    path("<int:id>/", views.product_detail, name="product_detail")
]
```

Grâce à cette configuration :

- /products/ affiche la liste des produits ;
- /products/1/ affiche le détail du produit ayant l'identifiant 1.

13 Partie 11 : Modifier le template layout.html

Votre projet possède déjà le fichier :

```
products/templates/layout.html
```

Ce fichier est important, car les pages `product_list.html` et `product_detail.html` l'utilisent déjà.

Il faut remplacer son contenu par la version suivante :

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}{% endblock %}</title>

    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/
        bootstrap.min.css"
        rel="stylesheet"
        integrity="sha384-EVSTQN3/
            azPrG1Anm3QDgpJLIm9Nao0Yz1ztcQTWfSpd3yD65VohhpuuC0mLASjC"
        crossorigin="anonymous">
</head>
```



```

<body>

<header>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">

      <a class="navbar-brand" href="{% url 'product_list' %}">
        E-Commerce
      </a>

      <button class="navbar-toggler" type="button"
        data-bs-toggle="collapse"
        data-bs-target="#navbarNavAltMarkup"
        aria-controls="navbarNavAltMarkup"
        aria-expanded="false"
        aria-label="Toggle navigation">

        <span class="navbar-toggler-icon"></span>
      </button>

      <div class="collapse navbar-collapse" id="navbarNavAltMarkup">

        <div class="navbar-nav">

          <a class="nav-link active" href="{% url 'product_list' %}">
            Accueil
          </a>

          <a class="nav-link" href="{% url 'product_list' %}">
            Produits
          </a>

          {% if user.is_authenticated %}

            <a class="nav-link" href="{% url 'profile' %}">
              Mon profil
            </a>

            <span class="navbar-text text-white mx-3">
              Bonjour, {{ user.username }}
            </span>

            <form method="post" action="{% url 'logout' %}"
              class="d-inline">
              {% csrf_token %}
              <button type="submit" class="btn btn-outline-light btn-sm">
                D connexion
              </button>
            </form>

          {% else %}

            <a class="nav-link" href="{% url 'login' %}">
              Connexion
            </a>

            <a class="nav-link" href="{% url 'signup' %}">
              Cr er un compte

```

```

        </a>

        {% endif %}

    </div>
</div>
</div>
</nav>
</header>

<main class="container mt-4">
    {% block content %}
    {% endblock %}
</main>

<footer class="text-center mt-5 mb-3">
    Copyright@EMI 2026
</footer>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/
    bootstrap.bundle.min.js"
    integrity="sha384-MrcW6ZMFYlzcLA8Nl+
    NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"
    crossorigin="anonymous">
</script>

</body>
</html>

```

Remarque

Dans ce projet, il ne faut pas créer un nouveau fichier `base.html`. On réutilise le fichier existant `layout.html`.

14 Partie 12 : Créer le dossier des templates de accounts

Dans l'application `accounts`, créer cette structure :

```

accounts/
    templates/
        registration/
            login.html
            signup.html
            profile.html

```

Sous Windows :

```

mkdir accounts\templates
mkdir accounts\templates\registration

```

Sous Linux ou macOS :

```

mkdir -p accounts/templates/registration

```

15 Partie 13 : Créer la page de connexion

Créer le fichier :

```
accounts/templates/registration/login.html
```

Ajouter le code suivant :

```
{% extends 'layout.html' %}

{% block title %}
Connexion
{% endblock %}

{% block content %}
<div class="row justify-content-center mt-5">
  <div class="col-md-5">

    <h2>Connexion</h2>

    <form method="post">
      {% csrf_token %}

      {{ form.as_p }}

      <button type="submit" class="btn btn-primary">
        Se connecter
      </button>
    </form>

    <p class="mt-3">
      Vous n'avez pas encore de compte ?
      <a href="{% url 'signup' %}">Cr éer un compte</a>
    </p>

  </div>
</div>
{% endblock %}
```

16 Partie 14 : Créer la page d'inscription

Créer le fichier :

```
accounts/templates/registration/signup.html
```

Ajouter le code suivant :

```
{% extends 'layout.html' %}

{% block title %}
Cr éer un compte
{% endblock %}

{% block content %}
<div class="row justify-content-center mt-5">
  <div class="col-md-6">

    <h2>Cr éation d'un compte</h2>

    <form method="post">
      {% csrf_token %}

      {{ form.as_p }}

    </form>
  </div>
</div>
{% endblock %}
```

```

        <button type="submit" class="btn btn-success">
            Cr e r mon compte
        </button>
    </form>

    <p class="mt-3">
        Vous avez d j un compte ?
        <a href="{% url 'login' %}">Se connecter</a>
    </p>

</div>
</div>
{% endblock %}

```

17 Partie 15 : Créer la page profil

Créer le fichier :

```
accounts/templates/registration/profile.html
```

Ajouter le code suivant :

```

{% extends 'layout.html' %}

{% block title %}
Mon profil
{% endblock %}

{% block content %}
<div class="mt-5">

    <h2>Mon profil</h2>

    <p>
        Cette page est prot g e . Elle est accessible uniquement aux
        utilisateurs connect s.
    </p>

    <h4>Informations du compte</h4>

    <ul class="list-group">
        <li class="list-group-item">
            Nom d'utilisateur : {{ user.username }}
        </li>

        <li class="list-group-item">
            Email : {{ user.email }}
        </li>

        <li class="list-group-item">
            Staff : {{ user.is_staff }}
        </li>

        <li class="list-group-item">
            Superutilisateur : {{ user.is_superuser }}
        </li>
    </ul>

```

```
</div>
{% endblock %}
```

18 Partie 16 : Vérifier les templates products

Votre fichier :

```
products/templates/product_list.html
```

peut rester comme suit :

```
{% extends 'layout.html' %}

{% block title %}
Liste de produits
{% endblock %}

{% block content %}
<ul>
    {% for p in products %}
        <li>
            {{ p.name }}
            {{ p.price }}
            {{ p.description }}

            {% if p.image %}
                
            {% endif %}

            <a href="{ url 'product_detail' p.id %}">D tails</a>
        </li>
    {% endfor %}
</ul>
{% endblock %}
```

Remarque

Il est préférable d'ajouter le test suivant avant d'afficher l'image :

```
{% if p.image %}
    
{% endif %}
```

Cela évite une erreur si un produit n'a pas d'image.

Votre fichier :

```
products/templates/product_detail.html
```

peut rester comme suit :

```
{% extends 'layout.html' %}

{% block title %}
D tail du produit
{% endblock %}

{% block content %}
<h3>{{ product.name }}</h3>
<h3>{{ product.price }}</h3>
```

```
<h3>{{ product.description }}</h3>

{% if product.image %}
    
{% endif %}

<br><br>

<a href="{% url 'product_list' %}">Retour</a>
{% endblock %}
```

19 Partie 17 : Exécuter les migrations

Même si la base de données `db.sqlite3` existe déjà, il faut exécuter les migrations pour s'assurer que les tables nécessaires à l'authentification existent.

Exécuter :

```
python manage.py migrate
```

Cette commande crée ou met à jour les tables suivantes :

- utilisateurs;
- groupes;
- permissions;
- sessions.

20 Partie 18 : Créer un superutilisateur

Si aucun compte administrateur n'existe encore, exécuter :

```
python manage.py createsuperuser
```

Django demande ensuite :

- le nom d'utilisateur;
- l'adresse email;
- le mot de passe.

21 Partie 19 : Lancer le serveur

Exécuter :

```
python manage.py runserver
```

Ouvrir ensuite le navigateur sur :

```
http://127.0.0.1:8000/
```

Cette adresse doit rediriger vers :

```
http://127.0.0.1:8000/products/
```

22 Partie 20 : Tester l'application

Travail demandé

Tester les scénarios suivants :

1. Accéder à la liste des produits :

```
http://127.0.0.1:8000/products/
```

2. Accéder au détail d'un produit :

```
http://127.0.0.1:8000/products/1/
```

3. Créer un nouveau compte :

```
http://127.0.0.1:8000/accounts/signup/
```

4. Vérifier que l'utilisateur est redirigé vers la page profil après l'inscription :

```
http://127.0.0.1:8000/accounts/profile/
```

5. Cliquer sur le bouton Déconnexion.

6. Se reconnecter :

```
http://127.0.0.1:8000/accounts/login/
```

7. Accéder à la page profil sans être connecté :

```
http://127.0.0.1:8000/accounts/profile/
```

8. Vérifier que Django redirige automatiquement vers la page de connexion.

23 Partie 21 : Explications importantes

23.1 Authentification

L'authentification permet de vérifier l'identité d'un utilisateur.

Exemple :

- l'utilisateur saisit son nom d'utilisateur ;
- il saisit son mot de passe ;
- Django vérifie les informations ;
- si les informations sont correctes, Django crée une session.

23.2 Session

Une session permet de garder l'utilisateur connecté pendant sa navigation.

Sans session, l'utilisateur devrait saisir son identifiant et son mot de passe à chaque page.

23.3 Mot de passe haché

Django ne stocke pas les mots de passe en clair dans la base de données.

Il stocke une version hachée du mot de passe. Cela améliore la sécurité de l'application.

23.4 Jeton CSRF

Le jeton CSRF protège les formulaires contre les attaques de type *Cross-Site Request Forgery*. Dans chaque formulaire utilisant la méthode POST, il faut ajouter :

```
{% csrf_token %}
```

23.5 Décorateur login_required

Le décorateur @login_required permet de protéger une vue.

Exemple :

```
from django.contrib.auth.decorators import login_required

@login_required
def profile(request):
    return render(request, "registration/profile.html")
```

Si l'utilisateur n'est pas connecté, Django le redirige vers la page définie par :

```
LOGIN_URL = "login"
```

24 Partie 22 : Intégration avec une fonctionnalité e-commerce

Dans un projet e-commerce, certaines fonctionnalités doivent être accessibles uniquement aux utilisateurs connectés.

Exemples :

- ajouter un produit au panier ;
- consulter son panier ;
- passer une commande ;
- consulter l'historique des commandes ;
- modifier son profil.

Exemple de protection d'une future vue panier dans products/views.py :

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render

@login_required
def cart(request):
    return render(request, "cart.html")
```

Travail demandé

Ajouter plus tard une page Panier protégée.

Tester ensuite :

1. l'accès au panier avec un utilisateur non connecté ;
2. l'accès au panier avec un utilisateur connecté.

25 Partie 23 : Travail à rendre

Chaque étudiant doit remettre :

1. le projet Django modifié ;
2. une capture de l'architecture après l'ajout de `accounts` ;
3. une capture de la liste des produits ;
4. une capture du détail d'un produit ;
5. une capture de la page d'inscription ;
6. une capture de la page de connexion ;
7. une capture de la page profil après connexion ;
8. une capture montrant la déconnexion ;
9. une capture de l'interface d'administration Django ;
10. une explication du rôle de `@login_required` ;
11. une explication du rôle de `{% csrf_token %}` ;
12. une démonstration que la page profil est inaccessible sans connexion.